

Python Master Notes

PART 2 — STRINGS & COLLECTIONS

Complete Reference for Interviews, Competitive Programming & AI/ML Foundations

PART 2: STRINGS & COLLECTIONS

1. STRINGS

1.1 String Creation

A string is an immutable, ordered sequence of Unicode characters.

```
s1 = 'hello'
s2 = "hello"
s3 = '''multi
line string'''
s4 = str(123)      # "123"
s5 = ""           # empty string
```

Single and double quotes are functionally identical — use double quotes when the string itself contains an apostrophe (`"it's ok"`), and vice versa. Triple quotes create multi-line strings and are also used for docstrings.

1.2 Indexing

```
s = "python"
s[0]      # 'p'
s[5]      # 'n'
s[6]      # IndexError: string index out of range
```

★ **MUST KNOW** Indexing starts at 0. The last valid index is `len(s) - 1`.

1.3 Negative Indexing

```
s = "python"
s[-1]     # 'n' (last char)
s[-2]     # 'o'
s[-6]     # 'p' (first char)
```

Negative indexing counts from the end: `-1` is always the last element, regardless of string length. This avoids writing `s[len(s) - 1]`.

1.4 Slicing

```
s = "python programming"
s[0:6]      # 'python'
s[:6]       # 'python' (start defaults to 0)
s[7:]       # 'programming' (end defaults to len(s))
s[::2]      # every 2nd char
s[::-1]     # reversed string
s[-11:]     # 'programming'
```

Syntax: `s[start:stop:step]` — `start` is inclusive, `stop` is exclusive, `step` defaults to 1. Slicing never raises an `IndexError` even for out-of-range bounds — it just clips.

🔗 **SHORTCUT** `s[::-1]` is the fastest, most Pythonic way to reverse a string.

1.5 String Immutability

Strings cannot be changed in place. Every "modification" creates a brand-new string object.

```
s = "hello"
s[0] = "H"      # ✗ TypeError: 'str' object does not support item assignment

s = "H" + s[1:]  # ✓ "Hello" — creates a new string
```

💡 **DEEP CONCEPT** Immutability means a string's identity (`id(s)`) is tied to its exact contents. This is why strings are hashable and can be used as dictionary keys, and why repeated concatenation in a loop (`result += s`) is $O(n^2)$ overall — each `+=`

builds an entirely new string. Prefer collecting pieces in a list and using `"".join(pieces)`, which is $O(n)$.

1.6 String Methods

`lower()`, `upper()`, `strip()`

```
"Hello".lower()      # 'hello'
"Hello".upper()      # 'HELLO'
"  hello  ".strip()  # 'hello'    (removes leading/trailing whitespace)
"xxhelloxx".strip("x") # 'hello'    (strips given characters, not a substring)
"  hi".lstrip()       # 'hi'       (left only)
"hi  ".rstrip()       # 'hi'       (right only)
```

⚠ COMMON MISTAKE `strip("xy")` removes any combination of the characters `x` and `y` from the ends — it is not a prefix/suffix removal tool. `"xyxhello".strip("xy")` gives `"hello"`, not `"hello"` after removing `"xyx"` as a block.

`replace()`

```
"banana".replace("a", "o")      # 'bonono'
"banana".replace("a", "o", 1)   # 'bonana'    (only first occurrence)
```

`find()` **VS** `index()`

```
"hello".find("l")      # 2    (first occurrence index)
"hello".find("z")      # -1   (not found, no error)
"hello".index("z")     # x ValueError: substring not found
```

🔗 INTERVIEW `find()` fails silently (returns -1); `index()` raises an exception. Choose based on whether "not found" is an expected case.

`count()`

```
"mississippi".count("s")      # 4
"aaaa".count("aa")           # 2    (non-overlapping matches)
```

`startswith()`, `endswith()`

```
"report.pdf".endswith(".pdf") # True
"IMG_2024.png".startswith("IMG") # True
"file.txt".endswith((".txt", ".csv")) # True — accepts a tuple of options
```

`split()`

```
"hello world python".split()      # ['hello', 'world', 'python'] (splits on any whitespace)
"a,b,,c".split(",")              # ['a', 'b', '', 'c'] (keeps empty strings)
"a,b,c".split(",", 1)            # ['a', 'b,c'] (maxsplit)
"line1\nline2".splitlines()       # ['line1', 'line2']
```

⚠ COMMON MISTAKE `"a,,b".split(",")` gives `['a', '', 'b']` — it does NOT collapse consecutive delimiters, unlike the no-argument `split()` which collapses all whitespace.

`join()`

```
" ".join(["hello", "world"])      # 'hello world'
".".join("abc")                   # 'a-b-c'    (joins characters of a string too)
```

★ MUST KNOW `split()` converts string → list; `join()` converts list → string, and is called *on the separator*, not the list.

`format()`

```
"{} is {} years old".format("Parth", 20)
"{name} scored {score}".format(name="A", score=90)
"{:.2f}".format(3.14159)          # '3.14'
"{:>10}".format("hi")             # right-align in width 10
"{:05d}".format(7)               # '00007'
```

f-strings

```
name, age = "Parth", 20
f"{name} is {age} years old"      # 'Parth is 20 years old'
```

```
f"{3.14159:.2f}"          # '3.14'
f"{10 + 5}"              # '15' (expressions allowed inside)
x = 5
f"{x=}"                  # 'x=5' (debug shortcut, Python 3.8+)
```

🔗 SHORTCUT f-strings are faster and more readable than `.format()` or `%`-formatting — prefer them in modern code and interviews.

1.7 String Membership

```
"ell" in "hello"        # True (substring check)
"z" not in "hello"      # True
```

The `in` operator on strings checks for substring presence and runs in $O(n)$ time.

1.8 String Comparison

```
"apple" == "apple"      # True (value equality)
"apple" < "banana"      # True (lexicographic, by Unicode code point)
"Apple" < "apple"       # True ('A' = 65, 'a' = 97 — uppercase sorts first)
sorted(["banana", "Apple", "cherry"]) # ['Apple', 'banana', 'cherry']
```

🔗 INTERVIEW String comparison is case-sensitive and character-by-character (like dictionary order, but based on ASCII/Unicode values). To compare ignoring case, use `a.lower() == b.lower()`.

1.9 Useful String Tricks

```
# Reverse
s[::-1]

# Palindrome check
s == s[::-1]

# Count vowels
sum(1 for c in s.lower() if c in "aeiou")

# Check anagram
sorted(s1) == sorted(s2)

# Remove all spaces
s.replace(" ", "")

# Check if all characters are alphanumeric
s.isalnum()

# Check if a string represents a number
s.isdigit() # only unsigned integers
s.isalpha() # only letters
```

🔗 SHORTCUT Building strings in a loop? Append pieces to a list and `"".join(list)` at the end instead of repeated `+=` — this avoids the $O(n^2)$ trap from string immutability.

1.10 Common String Interview Questions

Q1. Reverse the words in a sentence ("I love python" → "python love I")

```
" ".join(sentence.split()[::-1])
```

Q2. Check if two strings are anagrams

```
from collections import Counter
Counter(s1) == Counter(s2)
```

Q3. Find the first non-repeating character

```
from collections import Counter
freq = Counter(s)
```

```
for ch in s:
    if freq[ch] == 1:
        print(ch); break
```

Q4. Check if a string is a palindrome, ignoring case and spaces

```
clean = s.lower().replace(" ", "")
clean == clean[::-1]
```

Q5. Count occurrences of each character

```
from collections import Counter
Counter(s)
```

2. LISTS

2.1 Creating Lists

```
a = [1, 2, 3]
b = list()           # []
c = list("abc")      # ['a', 'b', 'c']
d = list(range(5))   # [0, 1, 2, 3, 4]
e = [0] * 5          # [0, 0, 0, 0, 0]
```

A list is an ordered, **mutable** collection that allows duplicates and can hold mixed types.

2.2 Indexing

```
a = [10, 20, 30]
a[0]      # 10
a[-1]     # 30
```

2.3 Slicing

```
a = [10, 20, 30, 40, 50]
a[1:4]     # [20, 30, 40]
a[:3]      # [10, 20, 30]
a[:: -1]   # [50, 40, 30, 20, 10]
a[:]       # full shallow copy
```

2.4 Updating

```
a = [10, 20, 30]
a[0] = 99      # [99, 20, 30]
a[0:2] = [1, 2, 3] # slice assignment can change the length: [1, 2, 3, 30]
```

DEEP CONCEPT Slice assignment can insert or remove multiple elements at once, since it replaces a whole sub-range rather than a single index.

2.5 Adding Elements

`append()`

```
a = [1, 2, 3]
a.append(4)      # [1, 2, 3, 4]
a.append([5, 6]) # [1, 2, 3, 4, [5, 6]] — adds as ONE element
```

`extend()`

```
a = [1, 2, 3]
a.extend([4, 5]) # [1, 2, 3, 4, 5] — adds elements individually
a.extend("ab")   # [1, 2, 3, 'a', 'b'] — iterates the argument
```

INTERVIEW `append(x)` always adds exactly one element (even if `x` is a list). `extend(iterable)` unpacks and adds each element of the iterable.

`insert()`

```
a = [1, 2, 3]
a.insert(1, 99) # [1, 99, 2, 3] — insert 99 at index 1
```

COMMON MISTAKE `insert(0, x)` is $O(n)$ — every existing element must shift right. Repeated inserts at the front are slow; use `collections.deque` if you need fast front insertion.

2.6 Removing Elements

`remove()`

```
a = [10, 20, 30, 20]
```

```
a.remove(20)      # removes FIRST occurrence: [10, 30, 20]
a.remove(99)      # ✗ ValueError: not in list
```

pop()

```
a = [10, 20, 30]
a.pop()          # removes & returns last element: 30; a = [10,20]
a.pop(0)         # removes & returns index 0: 10; a = [20]
```

clear()

```
a = [1, 2, 3]
a.clear()        # a = []
```

del

```
a = [10, 20, 30, 40]
del a[1]         # [10, 30, 40]
del a[0:2]       # deletes a slice
del a            # deletes the variable itself entirely
```

🔗 **INTERVIEW** `remove(value)` deletes by value; `pop(index)` and `del list[index]` delete by position. `pop()` returns the removed value, `del` does not.

2.7 sort() vs sorted()

```
a = [3, 1, 2]
a.sort()          # in-place, returns None, mutates a → [1,2,3]
b = sorted([3,1,2]) # returns a NEW list, original unchanged

a.sort(reverse=True) # descending
a.sort(key=len)      # sort by a key function
a.sort(key=lambda x: x[1]) # sort by second element of tuples
```

⚠ **COMMON MISTAKE** `a = a.sort()` sets `a` to `None`, because `sort()` mutates in place and returns `None`.

2.8 reverse()

```
a = [1, 2, 3]
a.reverse()       # in-place: [3, 2, 1]
b = a[::-1]       # new reversed list, non-destructive
```

2.9 copy()

```
a = [1, 2, 3]
b = a.copy()      # shallow copy — same as a[:] or list(a)
b.append(4)        # a unchanged, b = [1,2,3,4]
```

2.10 Nested Lists

```
matrix = [[1, 2], [3, 4]]
matrix[0][1] # 2
matrix[1].append(5) # [[1,2], [3,4,5]]
```

⚠ **COMMON MISTAKE — THE NESTED LIST TRAP**

```
bad = [[0] * 3] * 3
bad[0][0] = 1
print(bad)      # [[1,0,0], [1,0,0], [1,0,0]] ✗ all rows changed!
```

`[[0]*3]*3` creates **three references to the same inner list**, not three independent lists. Fix with a list comprehension, which creates a fresh inner list each time:

```
good = [[0] * 3 for _ in range(3)]
good[0][0] = 1
print(good)      # [[1,0,0], [0,0,0], [0,0,0]] ✓
```

2.11 List References

```
a = [1, 2, 3]
b = a          # b is the SAME object as a (a reference, not a copy)
b.append(4)
print(a)       # [1, 2, 3, 4] — a changed too!

c = a.copy()   # independent shallow copy
c.append(5)
print(a)       # unaffected

import copy
d = copy.deepcopy(a) # fully independent, even for nested structures
```

DEEP CONCEPT A shallow copy duplicates the outer list but the inner (nested) objects are still shared references. A deep copy recursively duplicates everything.

2.12 Common List Traps

- Mutable default arguments: `def f(x, arr=[]):` — the same list object persists across calls. Use `arr=None` and create a new list inside the function.
- Modifying a list while iterating over it (elements get skipped). Iterate over a copy (`for x in arr[:]:`) or build a new list instead.
- `a.append(x)` returns `None`, so `print(a.append(4))` prints `None`, not the updated list.
- `list1 * 2` repeats the whole list — `[1,2,3]*2 = [1,2,3,1,2,3]`, it does not double each element.

3. TUPLES

3.1 Creating Tuples

```
t1 = (1, 2, 3)
t2 = 1, 2, 3      # parentheses are optional
t3 = (5,)         # singleton tuple — the comma is REQUIRED
t4 = (5)          # this is just an int, NOT a tuple!
t5 = tuple([1, 2, 3]) # from a list
t6 = ()           # empty tuple
```

COMMON MISTAKE `(5)` is an integer in parentheses, not a tuple. A single-element tuple needs a trailing comma: `(5,)`.

3.2 Indexing / Slicing

```
t = (10, 20, 30, 40)
t[0]      # 10
t[-1]     # 40
t[1:3]    # (20, 30)
```

Indexing and slicing work exactly like lists — tuples only differ in mutability.

3.3 Immutability

```
t = (1, 2, 3)
t[0] = 99      # ✗ TypeError: 'tuple' object does not support item assignment
```

DEEP CONCEPT A tuple containing a mutable object (like a list) is still "immutable" in the sense that you can't reassign its slots — but the mutable object inside CAN change:

```
t = ([1, 2], 3)
t[0].append(99)    # ✓ allowed — the list inside changed, not the tuple's structure
print(t)           # ([1, 2, 99], 3)
```

3.4 Tuple Unpacking

```
point = (10, 20)
x, y = point          # x=10, y=20

a, *middle, b = [1, 2, 3, 4, 5]
# a = 1, middle = [2,3,4], b = 5

name, age = "Parth", 20      # simultaneous unpacking
a, b = b, a                  # classic swap trick, uses tuple packing/unpacking
```

3.5 When to Use Tuples

- Fixed collections of values that should never change (coordinates, RGB values, dates).
- As dictionary keys or set elements — tuples are hashable (if their contents are), lists are not.
- Returning multiple values from a function (Python packs them into a tuple automatically).
- Slightly faster and more memory-efficient than lists for fixed data.

3.6 Tuple Methods

Tuples support only two methods, since they cannot be modified:

```
t = (1, 2, 2, 3)
t.count(2)      # 2
t.index(3)      # 2 (index of first occurrence)
```

4. SETS

4.1 Creating Sets

```
s1 = {1, 2, 3}
s2 = set()          # empty set — {} creates a DICT, not a set!
s3 = set([1, 2, 2, 3]) # {1, 2, 3} — duplicates removed
s4 = set("hello")     # {'h','e','l','o'}
```

⚠ COMMON MISTAKE `{}` creates an empty **dictionary**, not an empty set. You must use `set()` for an empty set.

4.2 Uniqueness

```
arr = [1, 2, 2, 3, 3, 4]
unique = list(set(arr))    # [1, 2, 3, 4] (order not guaranteed)
```

💡 SHORTCUT To remove duplicates **while preserving order**, use `list(dict.fromkeys(arr))` instead of `set()`.

4.3 Adding / Removing

```
s = {1, 2, 3}
s.add(4)        # {1,2,3,4}
s.remove(2)     # {1,3,4} — raises KeyError if not present
s.discard(99)   # no error even if not present
s.pop()         # removes and returns an ARBITRARY element
s.clear()       # empty set
```

🔗 INTERVIEW `remove()` raises `KeyError` for a missing element; `discard()` silently does nothing. Prefer `discard()` when the element's presence isn't guaranteed.

4.4 Union

```
A = {1, 2, 3}
B = {3, 4, 5}
A | B          # {1,2,3,4,5}
A.union(B)     # same result
```

4.5 Intersection

```
A & B          # {3}
A.intersection(B)
```

4.6 Difference

```
A - B          # {1, 2} — in A but not B
B - A          # {4, 5} — in B but not A (not symmetric!)
```

4.7 Symmetric Difference

```
A ^ B          # {1, 2, 4, 5} — in exactly one of A or B
```

4.8 Membership

```
3 in A         # True — average O(1) lookup
```

★ MUST KNOW `x in set` is average O(1), while `x in list` is O(n). This makes sets the go-to structure for fast membership checks (e.g., Two Sum, duplicate detection).

4.9 Set Tricks

```
# Memory aid for operators
```

```
# | → OR → Union
# & → AND → Intersection
# - → Difference
# ^ → XOR → Symmetric difference

# Check duplicates in O(n)
len(arr) != len(set(arr))

# Frozen (immutable, hashable) set — usable as a dict key or set element
fs = frozenset([1, 2, 3])
```

4.10 Set Limitations

- Unordered — no guaranteed iteration order and **no indexing** (`s[0]` is invalid).
- Cannot contain duplicate or unhashable elements (no lists/dicts/sets as elements).
- Cannot be sliced.

5. DICTIONARIES

5.1 Key-Value Concept

A dictionary maps unique, hashable **keys** to values, implemented internally as a hash table for average $O(1)$ lookup, insertion, and deletion.

5.2 Creating Dictionaries

```
d1 = {"a": 1, "b": 2}
d2 = dict(a=1, b=2)
d3 = dict([("a", 1), ("b", 2)])
d4 = {} # empty dict
```

5.3 Accessing Values

```
d = {"a": 1, "b": 2}
d["a"] # 1
d["z"] # ✕ KeyError
d.get("z") # None (no error)
d.get("z", 0) # 0 (custom default)
```

5.4 Updating

```
d["a"] = 99 # updates existing key
d.update({"c": 3}) # adds/overwrites multiple keys
d.update(b=20) # keyword form
```

5.5 Adding / Removing

```
d["new"] = 5 # adds a new key
d.pop("new") # removes and returns the value
d.pop("missing", None) # returns default instead of raising KeyError
d.popitem() # removes & returns the LAST inserted (key, value) pair
del d["a"] # removes key, no return value
```

5.6 `keys()`, `values()`, `items()`

```
d = {"a": 1, "b": 2}
d.keys() # dict_keys(['a', 'b'])
d.values() # dict_values([1, 2])
d.items() # dict_items([('a', 1), ('b', 2)])
```

 **DEEP CONCEPT** These return dynamic "view objects" — if the dictionary changes afterward, the view reflects the change automatically.

5.7 Dictionary Iteration

```
for key in d:
    print(key)

for key, value in d.items():
    print(key, value)

for value in d.values():
    print(value)
```

5.8 Nested Dictionaries

```
students = {
    "A": {"age": 20, "grade": "A"},
    "B": {"age": 21, "grade": "B"}
}
```

```
students["A"]["age"]      # 20
students["A"]["age"] = 21  # update nested value
```

5.9 Dictionary Tricks

```
# Frequency counting (classic pattern)
freq = {}
for x in arr:
    freq[x] = freq.get(x, 0) + 1

# setdefault — get-or-initialize in one line
d = {}
d.setdefault("A", []).append(10)  # {"A": [10]}

# Dict comprehension
squares = {x: x*x for x in range(5)}

# Merging dictionaries (Python 3.9+)
merged = d1 | d2

# Merging (older syntax)
merged = {**d1, **d2}

# Invert a dictionary
inverted = {v: k for k, v in d.items()}
```

 **SHORTCUT** For pure frequency counting, `collections.Counter(arr)` is faster to write than the manual `get()` pattern.

5.10 Hashing Basics

DEEP CONCEPT

- A dictionary key must be **hashable** — it needs a stable `__hash__()` value for its lifetime. This means keys must be immutable types: `int`, `str`, `tuple` (if its contents are hashable), `frozenset`.
- Lists, sets, and dicts are unhashable and **cannot** be used as dict keys or set elements.
- Python computes `hash(key)` to decide which internal "bucket" stores the entry — this is what gives $O(1)$ average-case lookup, versus $O(n)$ for scanning a list.

```
d = {(1,2): "ok"}      # ✓ tuple key is fine
d = {[1,2]: "bad"}     # ✗ TypeError: unhashable type: 'list'
```

6. IMPORTANT CONCEPTS

6.1 Mutable vs Immutable

Simple explanation: Mutable objects can be changed in place after creation; immutable objects cannot — any "change" creates a new object.

```
Mutable:  list, dict, set
Immutable: int, float, bool, str, tuple, frozenset
```

```
a = [1, 2]
a.append(3)      # same object, modified in place
id(a)           # unchanged before/after

s = "hi"
s = s + "!"      # a NEW string object is created
```

⚠ **COMMON MISTAKE** Passing a mutable default argument: `def f(x, cache={}):` — the same dict is reused across every call, silently accumulating state. 🎯 **INTERVIEW** "Why are tuples hashable but lists aren't?" → Because hashability requires the value to never change; mutable objects can't guarantee a stable hash.

6.2 Iterable vs Non-Iterable

Simple explanation: An iterable is any object you can loop over with `for` — it implements `__iter__()` and returns an iterator.

```
Iterable:      list, tuple, str, dict, set, range, file objects, generators
Not iterable:  int, float, bool
```

```
for x in [1,2,3]: ... # ✓
for x in 5: ...      # ✗ TypeError: 'int' object is not iterable
```

💡 **DEEP CONCEPT** An **iterator** is a stateful object produced by `iter(iterable)`, which yields values one at a time via `next()` until it raises `StopIteration`. Every iterator is iterable, but not every iterable is an iterator (a list is iterable but not itself an iterator — you must call `iter()` on it first).

6.3 List vs Tuple

	List	Tuple
Mutable	Yes	No
Syntax	<code>[1, 2, 3]</code>	<code>(1, 2, 3)</code>
Hashable	No	Yes (if contents are hashable)
Performance	Slightly slower	Slightly faster, lower memory
Use case	Data that changes	Fixed records, dict keys

6.4 List vs Set

	List	Set
Ordered	Yes	No
Duplicates	Allowed	Not allowed
Indexing	Yes	No
Membership check	O(n)	O(1) average
Use case	Ordered sequence, duplicates matter	Uniqueness, fast lookup

6.5 Set vs Dictionary

	Set	Dictionary
Stores	Only values	Key-value pairs

Syntax	<code>{1, 2, 3}</code>	<code>{"a": 1}</code>
Empty literal	<code>set()</code>	<code>{}</code>
Lookup	By value, $O(1)$ avg	By key, $O(1)$ avg

DEEP CONCEPT Internally, both are built on the same hash-table mechanism — a set is essentially "a dict with no values."

6.6 String vs List

	String	List
Mutable	No	Yes
Element type	Only characters	Any type, mixed allowed
Concatenation cost	$O(n)$ per op $\rightarrow O(n^2)$ in a loop	Append is $O(1)$ amortized

SHORTCUT To build text piece-by-piece efficiently: collect characters/words in a list, then `"".join(list)` once at the end.

6.7 When to Use Each Collection

Need	Use
Ordered, changeable sequence	list
Fixed, protected sequence / dict key	tuple
Uniqueness + fast membership	set
Label $\rightarrow$ value mapping	dict
Text data	str

7. QUICK REVISION

- **Strings** — immutable; slice with `[start:stop:step]`; build with `join()`, not `+=` in loops; f-strings for formatting.
- **Lists** — mutable, ordered, allow duplicates; `append()` adds one element, `extend()` unpacks an iterable; watch the `[[0]*n]*n` shared-reference trap.
- **Tuples** — immutable, hashable (if contents are), used for fixed records and dict keys; singleton needs a trailing comma `(x,)`.
- **Sets** — unordered, unique elements, O(1) average membership; `{}` is a dict, not a set.
- **Dictionaries** — key→value hash table; use `.get()` to avoid `KeyError`; keys must be hashable.
- **Copying** — `b = a` is a reference; `b = a.copy()` is shallow; `copy.deepcopy(a)` is fully independent.
- **Mutable vs immutable** drives hashability, default-argument bugs, and whether something can be a dict key.

8. METHODS CHEAT SHEET

String Methods

Method	Purpose
<code>lower()</code> / <code>upper()</code>	Case conversion
<code>strip()</code> / <code>lstrip()</code> / <code>rstrip()</code>	Remove leading/trailing chars (default: whitespace)
<code>replace(old,new)</code>	Replace substring
<code>find()</code> / <code>index()</code>	Locate substring (−1 vs exception)
<code>count()</code>	Count non-overlapping occurrences
<code>startswith()</code> / <code>endswith()</code>	Prefix/suffix check
<code>split()</code> / <code>rsplit()</code> / <code>splitlines()</code>	String → list
<code>join()</code>	List → string
<code>format()</code> / f-string	String formatting
<code>isdigit()</code> / <code>isalpha()</code> / <code>isalnum()</code>	Content type checks

List Methods

Method	Purpose
<code>append(x)</code>	Add one element at the end — O(1)
<code>extend(iter)</code>	Add elements of an iterable
<code>insert(i,x)</code>	Insert at index — O(n)
<code>remove(x)</code>	Remove first matching value
<code>pop(i=-1)</code>	Remove & return by index (default: last)
<code>clear()</code>	Empty the list
<code>index(x)</code>	First index of value
<code>count(x)</code>	Occurrences of value
<code>sort(key=,reverse=)</code>	In-place sort
<code>reverse()</code>	In-place reverse
<code>copy()</code>	Shallow copy

Set Methods

Method	Purpose
<code>add(x)</code>	Add element
<code>remove(x)</code> / <code>discard(x)</code>	Remove (error / no error)
<code>pop()</code>	Remove arbitrary element
<code>union()</code> / <code> </code>	Combine sets
<code>intersection()</code> / <code>&</code>	Common elements
<code>difference()</code> / <code>-</code>	Elements only in first set
<code>symmetric_difference()</code> / <code>^</code>	Elements in exactly one set

Dictionary Methods

Method	Purpose
<code>get(k, default)</code>	Safe lookup
<code>keys()</code> / <code>values()</code> / <code>items()</code>	Views for iteration
<code>update(d2)</code>	Merge another dict in

<code>pop(k, default)</code>	Remove & return value
<code>popitem()</code>	Remove last inserted pair
<code>setdefault(k, default)</code>	Get, or set-then-get

9. 30 CONCEPTUAL QUESTIONS

1. Why are strings immutable in Python?

| For safety, hashability (so they can be dict keys), and internal performance optimizations like string interning.

2. What is the difference between `append()` and `extend()`?

| `append()` adds its argument as a single element; `extend()` iterates the argument and adds each element individually.

3. Why does `[[0]*3]*3` misbehave?

| It creates three references to the exact same inner list, so mutating one row mutates all of them.

4. What makes an object hashable?

| A stable `__hash__()` value that never changes during its lifetime — which requires the object to be immutable.

5. Difference between `is` and `==`?

| `==` compares values; `is` compares object identity (same memory location).

6. Why is `{}` a dictionary and not a set?

| Historically dict syntax came first; `{}` was reserved for the empty dict, so empty sets require the explicit `set()` call.

7. What is the time complexity of membership testing in a list vs a set?

| $O(n)$ for a list (linear scan) vs $O(1)$ average for a set (hash lookup).

8. Why can't a list be a dictionary key?

| Lists are mutable and therefore unhashable — their contents could change, invalidating a stored hash.

9. What is the difference between a shallow copy and a deep copy?

| A shallow copy duplicates the outer container but shares references to nested objects; a deep copy recursively duplicates everything.

10. What does slicing return when indices are out of range?

| It clips silently to valid bounds and never raises an `IndexError`, unlike direct indexing.

11. Why is `s += x` in a loop inefficient for strings?

| Because strings are immutable, each `+=` builds an entirely new string, making the total cost $O(n^2)$ for n concatenations.

12. What's the difference between `remove()` and `discard()` on a set?

| `remove()` raises `KeyError` if the element is absent; `discard()` does nothing silently.

13. What's the difference between `pop()` on a list vs a dict?

| On a list, `pop(index)` removes by position (default last). On a dict, `pop(key)` removes by key and needs a default to avoid `KeyError`.

14. Why does a tuple need a trailing comma for one element?

| Parentheses alone are just grouping syntax; the comma is what actually signals "this is a tuple."

15. What is a view object (e.g. from `dict.keys()`)?

| A dynamic, live window onto the dictionary's data — it reflects future changes to the dict automatically rather than being a frozen snapshot.

16. Why do sets have no guaranteed order?

| Elements are stored by their hash value's bucket position, not insertion order, so iteration order depends on internal hashing, not sequence.

17. Can a tuple be mutable in any sense?

| The tuple's own structure can't change, but if it holds a mutable object (like a list), that inner object can still be modified.

18. What's the difference between `.sort()` and `sorted()` ?

| `.sort()` mutates the list in place and returns `None`; `sorted()` returns a new list, leaving the original unchanged.

19. Why is `insert(0, x)` considered slow for large lists?

| Because every existing element must shift one position to the right — an $O(n)$ operation.

20. How do you safely access a dictionary key that might not exist?

| Use `d.get(key, default)` instead of `d[key]`, which raises `KeyError` if missing.

21. What's the difference between `list()` and `[]` for creating an empty list?

| Functionally identical; `[]` is the idiomatic, slightly faster literal form.

22. Why is `frozenset` useful?

| It's an immutable, hashable version of a set, so it can be used as a dict key or as an element of another set.

23. What happens when you iterate over a dictionary directly?

| You iterate over its keys, in insertion order (guaranteed since Python 3.7).

24. Why is negative indexing useful?

| It lets you access elements relative to the end without needing to know the collection's length.

25. What's the danger of modifying a list while looping over it?

| Elements can be skipped or an index error can occur, because the list's length and positions shift mid-iteration.

26. Difference between `find()` and `index()` on strings?

| `find()` returns -1 if not found; `index()` raises `ValueError`.

27. What does "hashable" mean in relation to set elements?

| Only hashable (immutable) values can be stored in a set — lists, dicts, and other sets cannot be set elements.

28. What is the difference between `split()` with no arguments and `split(" ")` ?

| No-argument `split()` splits on any whitespace and collapses consecutive separators; `split(" ")` splits strictly on single spaces and keeps empty strings between consecutive spaces.

29. Why use `Counter` instead of a manual frequency dictionary?

| It's more concise, comes with helper methods like `most_common()`, and defaults missing keys to 0 automatically.

30. What's the relationship between mutability and default function arguments?

| A mutable default argument (like `[]` or `{}`) is created only once at function definition time and shared across all calls, causing hidden state to accumulate — the standard fix is to default to `None` and create the object inside the function.

10. 30 OUTPUT-PREDICTION QUESTIONS

1.

```
a = [1,2,3]
b = a
b.append(4)
print(a)
```

`[1, 2, 3, 4]` — b references the same list as a.

2.

```
a = [1,2,3]
b = a.copy()
b.append(4)
print(a)
```

`[1, 2, 3]` — copy() creates an independent outer list.

3.

```
print([1,2,3] * 2)
```

`[1, 2, 3, 1, 2, 3]` — repeats the whole list, not each element.

4.

```
print("ab" * 3)
```

`abababab` ... → actually `'ababab'` .

5.

```
a = [1,2,3]
print(a.append(4))
```

`None` — append() mutates in place and returns None.

6.

```
print([1,2,3].pop())
```

`3`

7.

```
a = [1,2,3]
print(a[::-1])
```

`[3, 2, 1]`

8.

```
print(bool([]), bool([1]))
```

`False True`

9.

```
a = [1,2]; b = [1,2]
print(a == b, a is b)
```

`True False` — equal values, different objects.

10.

```
t = (1,2,3)
t[0] = 5
```

`TypeError: 'tuple' object does not support item assignment`

11.

```
t = (5)
print(type(t))
```

`<class 'int'>` — no comma, so it's not a tuple.

12.

```
a, *b = [1,2,3,4]
print(a, b)
```

`1 [2, 3, 4]`

13.

```
bad = [[0]*2]*2
bad[0][0] = 9
print(bad)
```

`[[9, 0], [9, 0]]` — shared inner-list reference trap.

14.

```
s = {1,2,2,3}
print(s)
```

`{1, 2, 3}` — duplicates removed.

15.

```
d = {}
print(type(d))
```

`<class 'dict'>` — not a set.

16.

```
print("hello".find("z"))
```

`-1`

17.

```
print("hello".index("z"))
```

`ValueError: substring not found`

18.

```
a = "hello"
a[0] = "H"
```

`TypeError: 'str' object does not support item assignment`

19.

```
print(sorted(["banana","Apple","cherry"]))
```

`['Apple', 'banana', 'cherry']` — uppercase letters sort before lowercase.

20.

```
a = [3,1,2]
b = a.sort()
print(b)
```

`None` — `sort()` returns `None`.

21.

```
A = {1,2,3}
B = {2,3,4}
print(A - B)
```

```
{1}
```

22.

```
A = {1,2,3}
B = {2,3,4}
print(A ^ B)
```

```
{1, 4}
```

23.

```
d = {"a":1}
print(d.get("b", 0))
```

```
0
```

24.

```
d = {"a":1}
print(d["b"])
```

```
KeyError: 'b'
```

25.

```
x = 10
def f():
    x = 20
    print(x)
f()
print(x)
```

```
20
```

 then

```
10
```

 — the local x shadows the global one inside f().

26.

```
print("a,,b".split(","))
```

```
['a', '', 'b']
```

 — empty strings are preserved.

27.

```
print(" ".join(['1','2','3']))
```

```
1 2 3
```

28.

```
a = (1, [2,3])
a[1].append(4)
print(a)
```

```
(1, [2, 3, 4])
```

 — the inner list is mutable even though the tuple itself is not.

29.

```
print(list({3,1,2}) == [1,2,3])
```

Not guaranteed — set iteration order is not sorted order, so this may be `False` even though the elements match. (In current CPython for small ints it often prints True, but this is an implementation detail, not a language guarantee.)

30.

```
a = [1,2,3]
for x in a:
    if x == 2:
        a.remove(x)
print(a)
```

`[1, 3]` — but this relies on unsafe in-place mutation during iteration; for larger cases elements can be skipped, so this pattern should be avoided.

11. 30 CODING PROBLEMS WITH COMPLETE SOLUTIONS

1. Reverse a string.

```
def reverse_str(s):  
    return s[::-1]
```

2. Check if a string is a palindrome.

```
def is_palindrome(s):  
    s = s.lower().replace(" ", "")  
    return s == s[::-1]
```

3. Count vowels in a string.

```
def count_vowels(s):  
    return sum(1 for c in s.lower() if c in "aeiou")
```

4. Check if two strings are anagrams.

```
from collections import Counter  
def is_anagram(s1, s2):  
    return Counter(s1) == Counter(s2)
```

5. Find the first non-repeating character.

```
from collections import Counter  
def first_unique(s):  
    freq = Counter(s)  
    for ch in s:  
        if freq[ch] == 1:  
            return ch  
    return None
```

6. Reverse the words in a sentence.

```
def reverse_words(s):  
    return " ".join(s.split()[::-1])
```

7. Remove duplicates from a list, preserving order.

```
def dedup(arr):  
    return list(dict.fromkeys(arr))
```

8. Find the second largest element in a list.

```
def second_largest(arr):  
    uniq = sorted(set(arr))  
    return uniq[-2] if len(uniq) >= 2 else None
```

9. Flatten a nested list (one level deep).

```
def flatten(nested):  
    return [x for sub in nested for x in sub]
```

10. Find the intersection of two lists.

```
def intersect(a, b):  
    return list(set(a) & set(b))
```

11. Rotate a list left by k positions.

```
def rotate_left(arr, k):  
    k %= len(arr)
```

```
return arr[k:] + arr[:k]
```

12. Find all pairs in a list that sum to a target (Two Sum).

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return (seen[need], i)
        seen[x] = i
    return None
```

13. Count the frequency of each element in a list.

```
from collections import Counter
def frequency(arr):
    return Counter(arr)
```

14. Find the most frequent element in a list.

```
from collections import Counter
def most_frequent(arr):
    return Counter(arr).most_common(1)[0][0]
```

15. Merge two dictionaries.

```
def merge_dicts(d1, d2):
    return {**d1, **d2} # or d1 | d2 in Python 3.9+
```

16. Invert a dictionary (swap keys and values).

```
def invert(d):
    return {v: k for k, v in d.items()}
```

17. Group words by their length using a dictionary.

```
from collections import defaultdict
def group_by_length(words):
    groups = defaultdict(list)
    for w in words:
        groups[len(w)].append(w)
    return dict(groups)
```

18. Check if a list contains any duplicates.

```
def has_duplicates(arr):
    return len(arr) != len(set(arr))
```

19. Find the union and intersection of two sets.

```
def set_ops(a, b):
    return a | b, a & b
```

20. Check if one string is a rotation of another.

```
def is_rotation(s1, s2):
    return len(s1) == len(s2) and s2 in (s1 + s1)
```

21. Find all unique pairs of elements from two lists (Cartesian pairs) that share no common value.

```
def symmetric_pairs(a, b):
    return list(set(a) ^ set(b))
```

22. Sort a list of tuples by the second element.

```
def sort_by_second(pairs):
    return sorted(pairs, key=lambda x: x[1])
```

23. Count word frequency in a sentence.

```
from collections import Counter
def word_freq(sentence):
    return Counter(sentence.lower().split())
```

24. Implement a stack using a list (push, pop, peek).

```
class Stack:
    def __init__(self):
        self.data = []
    def push(self, x):
        self.data.append(x)
    def pop(self):
        return self.data.pop()
    def peek(self):
        return self.data[-1]
```

25. Implement a queue using `collections.deque`.

```
from collections import deque
class Queue:
    def __init__(self):
        self.data = deque()
    def enqueue(self, x):
        self.data.append(x)
    def dequeue(self):
        return self.data.popleft()
```

26. Check if two lists are permutations of each other.

```
from collections import Counter
def is_permutation(a, b):
    return Counter(a) == Counter(b)
```

27. Find the longest word in a sentence.

```
def longest_word(sentence):
    return max(sentence.split(), key=len)
```

28. Deep-copy a nested list and verify independence.

```
import copy
def independent_copy(nested):
    return copy.deepcopy(nested)
```

29. Find the missing number in a list of 1..n.

```
def missing_number(arr, n):
    return n * (n + 1) // 2 - sum(arr)
```

30. Given a dictionary of student:score, return the top 3 students by score.

```
def top_students(scores, k=3):
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)[:k]
```

12. ONE-PAGE COLLECTION CHEAT SHEET

Type	Ordered	Mutable	Duplicates	Indexed	Hashable	Literal
str	Yes	No	Yes	Yes	Yes	<code>"abc"</code>
list	Yes	Yes	Yes	Yes	No	<code>[1, 2, 3]</code>
tuple	Yes	No	Yes	Yes	Yes*	<code>(1, 2, 3)</code>
set	No	Yes	No	No	No	<code>{1, 2, 3}</code>
frozenset	No	No	No	No	Yes	<code>frozenset(...)</code>
dict (keys)	Yes (insertion order)	Yes	Keys: No	By key	Keys must be hashable	<code>{"a": 1}</code>

* Tuple is hashable only if every element inside it is also hashable.

Common Operation Complexity

Operation	List	Set	Dict
Access by index/key	$O(1)$	—	$O(1)$ avg
Membership (<code>in</code>)	$O(n)$	$O(1)$ avg	$O(1)$ avg (keys)
Append / add	$O(1)$ amortized	$O(1)$ avg	$O(1)$ avg
Insert at front	$O(n)$	—	—
Delete by value	$O(n)$	$O(1)$ avg	$O(1)$ avg
Sort	$O(n \log n)$	—	—

Golden Rules

- ★ Strings and tuples are immutable → any "modification" makes a new object.
- ★ Lists, sets, and dicts are mutable → beware shared references (`b = a`) and the nested-list trap.
- 🔍 Need fast lookup / uniqueness → set. Need order + duplicates → list. Need labeled data → dict. Need a fixed record/dict key → tuple.
- 🔍 `.get()` over `d[key]`, `discard()` over `remove()`, `Counter` over manual frequency dicts — safer, shorter interview code.
- 🔒 Hashability = immutability. Only hashable objects can be dict keys or set elements.